

CYBERSTER



WEEK 01 TECHNICAL REPORT

**Virtual SOC Environment Setup using VMware, Wireshark, Wazuh SIEM,
Threat Hunting, and MITRE ATT&CK Framework**

Prepared By
Farukh Riaz

Submitted To
Abdullah Zia

Internship Domain

Security Operations Center (SOC)

Introduction

Cybersecurity plays a vital role in protecting computer systems and networks from unauthorized access, cyberattacks, and data breaches. As organizations continue to rely on digital technologies, the need for effective security monitoring and incident detection has become increasingly important. Security Information and Event Management (SIEM) solutions help security teams monitor network activities, analyze security events, and respond to potential threats in real time.

During this week's practical lab, a complete virtual cybersecurity environment was developed using VMware Workstation. The lab consisted of an Ubuntu virtual machine configured as the Wazuh SIEM server, a Kali Linux virtual machine used for security testing, and a Windows machine configured as a monitored endpoint. The objective of building this environment was to understand how different systems communicate within a network, how network traffic can be analyzed using Wireshark, and how Wazuh collects, monitors, and analyzes security events from connected agents.

Throughout the practical exercises, virtual networking was configured, communication between machines was verified, packet capturing was performed using Wireshark, and Wazuh agents were successfully deployed on both Windows and Kali Linux. Finally, the Wazuh dashboard was used to monitor endpoints, analyze generated security events, and gain practical experience with basic Security Operations Center (SOC) activities.

This lab provided practical knowledge of network monitoring, packet analysis, endpoint monitoring, and SIEM deployment, which are fundamental concepts in modern cybersecurity operations.

Overview of Week 1

The first week of the Cyberster internship was focused on setting up a complete virtual cybersecurity laboratory and understanding the fundamentals of network monitoring and Security Information and Event Management (SIEM). The primary objective was to build a secure lab environment where different operating systems could communicate with each other and generate security events for monitoring and analysis.

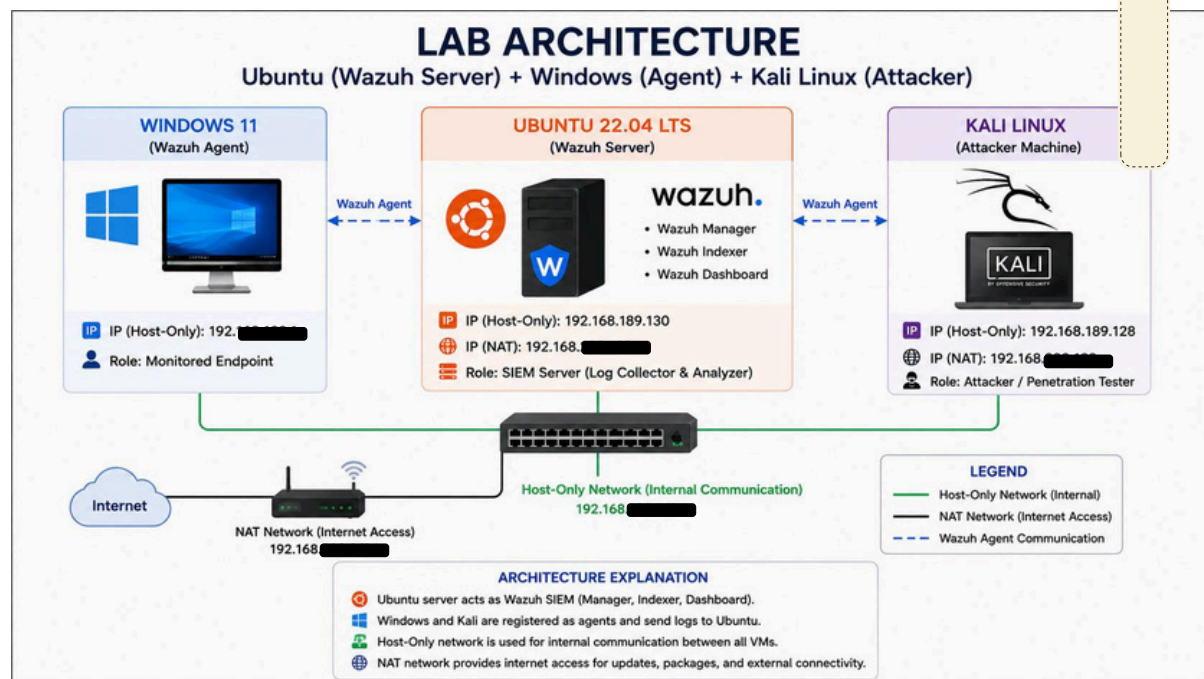
The practical work started with configuring VMware Workstation and creating the required virtual machines. Ubuntu Linux was deployed as the central Wazuh SIEM server, Kali Linux was prepared as the penetration testing machine, and a Windows system was configured as the monitored endpoint. Virtual networking was established using NAT and Host-Only

adapters to provide both internet connectivity and isolated communication between the machines.

After configuring the virtual environment, network connectivity was verified by assigning appropriate IP addresses and testing communication between all systems using ICMP ping. Once the network was functioning correctly, Wireshark was installed to capture and analyze network traffic. Different protocols such as ARP, ICMP, DNS, TCP, and HTTPS were observed to understand how devices communicate over the network.

The next phase of the lab involved deploying the Wazuh SIEM platform on Ubuntu Linux. After the successful installation of the Wazuh server, agents were installed and configured on both the Windows and Kali Linux systems. These agents were connected to the Wazuh manager, allowing the dashboard to receive endpoint logs and security events in real time. Finally, the Wazuh dashboard was used to verify the connected agents, monitor system activities, and explore different security monitoring features such as Security Events, Threat Hunting, and MITRE ATT&CK mapping. By the end of the week, a fully functional SOC lab environment had been successfully established, providing hands-on experience with network analysis, endpoint monitoring, and security event management.

Lab Architecture



The figure illustrates the overall architecture of the virtual cybersecurity laboratory. Ubuntu Linux hosts the Wazuh SIEM server, while Windows and Kali Linux operate as monitored endpoints through Wazuh agents. The Host-Only network enables secure communication between all virtual machines, whereas the NAT adapter provides internet connectivity for software installation and updates.

Explanation

To perform the practical activities of Week 1, a virtual cybersecurity laboratory was designed using VMware Workstation. The architecture consisted of three main systems connected through VMware virtual networking to simulate a small Security Operations Center (SOC) environment. Each machine had a specific role in the overall setup, allowing communication, monitoring, and security analysis within an isolated network.

The Ubuntu virtual machine served as the Wazuh SIEM server and acted as the central management system. It was responsible for collecting logs, processing security events, and displaying monitoring information through the Wazuh Dashboard. Since all security-related data was forwarded to this server, it became the core component of the entire lab environment.

The Windows machine was configured as a monitored endpoint by installing the Wazuh Agent. This agent continuously collected system logs and security events and securely transmitted them to the Wazuh server. These events were later analyzed through the Wazuh Dashboard to observe endpoint activities and detect potential security issues.

Kali Linux was deployed as the security testing machine. It was used to perform various networking and security-related activities such as network scanning, connectivity testing, and generating traffic that could be monitored by the Wazuh server. This allowed practical observation of how security events are generated and recorded inside a SIEM environment. The networking environment was configured using two VMware virtual adapters. The NAT adapter provided internet connectivity for downloading software packages, updates, and dependencies. The Host-Only adapter created an isolated internal network that enabled secure communication between Ubuntu, Kali Linux, and the Windows machine without exposing the laboratory environment to the external network.

As a result, all three systems were able to communicate successfully while maintaining a controlled and secure testing environment suitable for cybersecurity experimentation.

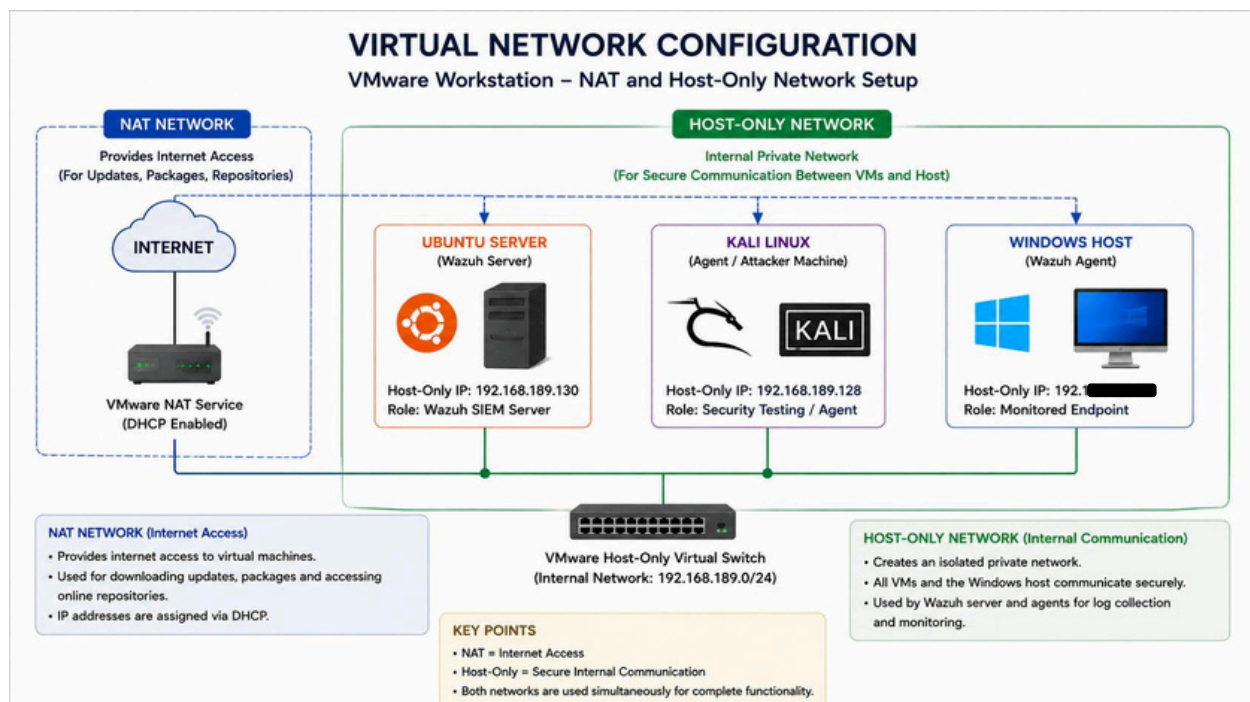
Network Configuration

VirtualNetworkConfiguration

To establish communication between all systems in the cybersecurity lab, VMware Workstation was configured using two virtual network adapters: **NAT** and **Host-Only**. This configuration provided internet connectivity for downloading software packages while also creating an isolated internal network for secure communication between the virtual machines.

The NAT adapter was primarily used to provide internet access to both Ubuntu and Kali Linux. This allowed the systems to install updates, download required packages, and access online repositories during software installation.

The Host-Only adapter created a private network that connected Ubuntu, Kali Linux, and the Windows host. This isolated network enabled secure communication between all machines without exposing the laboratory environment to the external network. Since Wazuh relies on continuous communication between the server and its agents, the Host-Only network played a crucial role in the successful deployment of the monitoring infrastructure.



Ubuntu Network Configuration

The Ubuntu virtual machine was configured with **two network interfaces**, each serving a different purpose.

The **ens33** interface was connected to the NAT network and obtained its IP address automatically through DHCP. This interface provided internet connectivity required for installing the Wazuh platform and downloading software packages.

The **ens34** interface was connected to the Host-Only network. Initially, it also received an IP address through DHCP; however, to ensure consistent communication with the Wazuh agents, it was later configured with a **static IP address**. Assigning a static IP ensured that the Windows and Kali agents could always communicate with the Wazuh server using the same address.

The network configuration was verified using the `ip a` and `ip route` commands to confirm that both interfaces were active and correctly configured

```
alpha@alpha:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:83:29:fb brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.127.128/24 metric 100 brd 192.168.127.255 scope global dynamic ens33
        valid_lft 1234sec preferred_lft 1234sec
    inet6 fe80::20c:29ff:fe83:29fb/64 scope link
        valid_lft forever preferred_lft forever
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:83:29:05 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.189.128/24 metric 100 brd 192.168.189.255 scope global dynamic ens34
        valid_lft 1235sec preferred_lft 1235sec
    inet6 fe80::20c:29ff:fe83:2905/64 scope link
        valid_lft forever preferred_lft forever
alpha@alpha:~$ _
```

Kali Linux Network Configuration

The Kali Linux virtual machine was also configured with two network adapters.

The first interface was connected to the NAT network to provide internet access, while the second interface was connected to the Host-Only network for communication with the Ubuntu Wazuh server.

Unlike the Ubuntu server, Kali Linux continued using dynamically assigned IP addresses because it was only functioning as a monitored endpoint and testing machine. A static IP configuration was not required for the objectives of this lab.

The configuration was verified using the `ip a` command, confirming that both interfaces were active and operating correctly.

```
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:f6:da:c3 brd ff:ff:ff:ff:ff:ff
   inet 192.168.127.129/24 brd 192.168.127.255 scope global dynamic noprefixroute eth0
       valid_lft 1760sec preferred_lft 1760sec
   inet6 fe80::1f0f:1cef:c151:b016/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:f6:da:cd brd ff:ff:ff:ff:ff:ff
   inet 192.168.189.130/24 brd 192.168.189.255 scope global dynamic noprefixroute eth1
       valid_lft 1760sec preferred_lft 1760sec
   inet6 fe80::5204:58e5:8c4f:1f03/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
(kali@kali)-[~]
```

Windows Host Network Configuration

The Windows host machine communicated with the virtual machines through the VMware Host-Only network adapter. This adapter was automatically assigned an IP address within the same subnet as the Ubuntu and Kali Host-Only interfaces, allowing direct communication between all systems.

The Windows machine also maintained its normal Wi-Fi connection for internet access. As a result, the host had two active network connections: one for internet access and another dedicated to the isolated laboratory network.

The network configuration was verified using the `ipconfig` command.

```

Wireless LAN adapter Local Area Connection* 10:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Ethernet adapter VMware Network Adapter VMnet1:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::9f6:e1ab:db5:
    IPv4 Address. . . . . : 192.168.189.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Ethernet adapter VMware Network Adapter VMnet8:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::624:35ca:
    IPv4 Address. . . . . : 192.168.
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Wireless LAN adapter Wi-Fi:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::b0b0:
    IPv4 Address. . . . . : 192.168.
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : fe80::1%18
                                192.168.

```

Network Connectivity Verification

After completing the network configuration, connectivity tests were performed between Ubuntu, Kali Linux, and the Windows host using the ICMP Ping utility.

The test results confirmed successful communication between Ubuntu and Kali Linux, as well as between the Windows host and both virtual machines. This verified that the Host-Only network was functioning correctly and that all systems were able to exchange data before deploying the Wazuh monitoring platform.

These successful connectivity tests ensured that the environment was fully prepared for packet analysis using Wireshark and for the deployment of the Wazuh SIEM infrastructure.

Ping Kali from Windows

```
C:\Users\chfar>ping 192.168.189.130

Pinging 192.168.189.130 with 32 bytes of data:
Reply from 192.168.189.130: bytes=32 time=2ms TTL=64
Reply from 192.168.189.130: bytes=32 time=1ms TTL=64
Reply from 192.168.189.130: bytes=32 time<1ms TTL=64
Reply from 192.168.189.130: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.189.130:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 2ms, Average = 0ms

C:\Users\chfar>
```

Ping Windows from kali

```
(kali㉿kali)-[~]
$ ping 192.168.189.1
PING 192.168.189.1 (192.168.189.1) 56(84) bytes of data.
64 bytes from 192.168.189.1: icmp_seq=1 ttl=128 time=0.424 ms
64 bytes from 192.168.189.1: icmp_seq=2 ttl=128 time=1.80 ms
64 bytes from 192.168.189.1: icmp_seq=3 ttl=128 time=0.326 ms
64 bytes from 192.168.189.1: icmp_seq=4 ttl=128 time=0.817 ms
64 bytes from 192.168.189.1: icmp_seq=5 ttl=128 time=1.02 ms
^C
— 192.168.189.1 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4046ms
rtt min/avg/max/mdev = 0.326/0.877/1.798/0.525 ms
```

Ping Ubuntu From Windows

```
C:\Users\chfar>ping 192.168.189.128

Pinging 192.168.189.128 with 32 bytes of data:
Reply from 192.168.189.128: bytes=32 time<1ms TTL=64
Reply from 192.168.189.128: bytes=32 time=3ms TTL=64
Reply from 192.168.189.128: bytes=32 time<1ms TTL=64
Reply from 192.168.189.128: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.189.128:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 3ms, Average = 0ms
```

Ping Windows From Ubuntu

```
alpha@alpha:~$ ping 192.168.189.1
PING 192.168.189.1 (192.168.189.1) 56(84) bytes of data.
64 bytes from 192.168.189.1: icmp_seq=1 ttl=128 time=0.413 ms
64 bytes from 192.168.189.1: icmp_seq=2 ttl=128 time=0.514 ms
64 bytes from 192.168.189.1: icmp_seq=3 ttl=128 time=0.432 ms
64 bytes from 192.168.189.1: icmp_seq=4 ttl=128 time=0.944 ms
^C
--- 192.168.189.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3029ms
rtt min/avg/max/mdev = 0.413/0.575/0.944/0.215 ms
alpha@alpha:~$ _
```

Ping Ubuntu From kali

```
(kali@kali)-[~]
$ ping 192.168.189.128
PING 192.168.189.128 (192.168.189.128) 56(84) bytes of data.
64 bytes from 192.168.189.128: icmp_seq=1 ttl=64 time=0.597 ms
64 bytes from 192.168.189.128: icmp_seq=2 ttl=64 time=0.775 ms
64 bytes from 192.168.189.128: icmp_seq=3 ttl=64 time=1.97 ms
64 bytes from 192.168.189.128: icmp_seq=4 ttl=64 time=1.84 ms
64 bytes from 192.168.189.128: icmp_seq=5 ttl=64 time=0.759 ms
^C
— 192.168.189.128 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4054ms
rtt min/avg/max/mdev = 0.597/1.188/1.974/0.590 ms
```

Ping kali From Ubuntu

```
alpha@alpha:~$ ping 192.168.189.130
PING 192.168.189.130 (192.168.189.130) 56(84) bytes of data.
64 bytes from 192.168.189.130: icmp_seq=1 ttl=64 time=0.681 ms
64 bytes from 192.168.189.130: icmp_seq=2 ttl=64 time=1.19 ms
64 bytes from 192.168.189.130: icmp_seq=3 ttl=64 time=0.924 ms
64 bytes from 192.168.189.130: icmp_seq=4 ttl=64 time=1.98 ms
^C
--- 192.168.189.130 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 0.681/1.194/1.978/0.487 ms
alpha@alpha:~$ _
```

Static IP Configuration on Ubuntu Server

To ensure reliable communication between the Wazuh server and its connected agents, a static IP address was assigned to the Ubuntu virtual machine. Initially, both network interfaces were configured to obtain IP addresses dynamically through DHCP. However,

since the Wazuh Manager must always remain reachable at a fixed address, the Host-Only interface was configured with a static IP.

A static IP address eliminates the possibility of IP changes after system reboot or network restart. This is essential because the Windows and Kali Linux agents communicate with the Wazuh server using its IP address. If the server's IP changes, the agents would fail to establish a connection until their configuration is updated.

The network configuration file was modified using Ubuntu's Netplan utility. The Host-Only interface (**ens34**) was assigned a permanent IP address, while the NAT interface (**ens33**) continued to receive its address dynamically to maintain internet connectivity.

After saving the configuration, the changes were applied successfully using the Netplan configuration utility. The updated network settings were verified using the `ip a` and `ip route` commands, confirming that the static IP address had been assigned correctly and that internet connectivity remained functional.

The successful implementation of the static IP configuration ensured stable communication between the Wazuh server and all connected agents throughout the laboratory exercises.

Checking Ip Route of Ubuntu



```
kali-linux-2025.4-vmware-am... X Ubuntu 64-bit X Windows 10 x64 X
alpha@alpha:~$ ip route
default via 192.168.127.2 dev ens33 proto dhcp src 192.168.127.128 metric 100
192.168.127.0/24 dev ens33 proto kernel scope link src 192.168.127.128 metric 100
192.168.127.2 dev ens33 proto dhcp scope link src 192.168.127.128 metric 100
192.168.189.0/24 dev ens34 proto kernel scope link src 192.168.189.128 metric 100
192.168.189.1 dev ens34 proto dhcp scope link src 192.168.189.128 metric 100
alpha@alpha:~$
alpha@alpha:~$ sudo nano /etc/netplan/50-cloud-init.yaml
```

Netplan Configuration Before Editing

The default Netplan configuration file was configured to obtain IP addresses dynamically using DHCP for both network interfaces (ens33 and ens34). While this setup provided network connectivity, it was not appropriate for a server that required a permanent IP address.

```
GNU nano 7.2
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: true
    ens34:
      dhcp4: true
```

Netplan Configuration After Editing

The Netplan configuration file was modified to assign a static IP address to the ens34 interface while keeping the ens33 interface configured for DHCP. This allowed the Ubuntu server to maintain internet access through the NAT network while providing a fixed IP address for communication with the Wazuh agents.

```
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: true

    ens34:
      dhcp4: false
      addresses:
        - 192.168.189.10/24
```

Applying the Netplan Configuration

After saving the updated configuration file, the changes were applied using the Netplan utility. The command completed successfully without generating any errors, indicating that the new network configuration had been accepted by the operating system.

```
alpha@alpha:~$ sudo netplan apply
alpha@alpha:~$
```

4.7: Verification of the Static IP Address

The ip a command was executed to verify the updated network configuration. The output confirmed that the ens34 interface had been successfully assigned the static IP address **192.168.189.10**, while the ens33 interface continued to receive its IP address dynamically from the NAT network.

```
alpha@alpha:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:83:29:fb brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.127.128/24 metric 100 brd 192.168.127.255 scope global dynamic ens33
        valid_lft 1739sec preferred_lft 1739sec
    inet6 fe80::20c:29ff:fe83:29fb/64 scope link
        valid_lft forever preferred_lft forever
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:83:29:05 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.189.10/24 brd 192.168.189.255 scope global ens34
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe83:2905/64 scope link
        valid_lft forever preferred_lft forever
alpha@alpha:~$
```

Pinging From kali

```
(kali㉿kali)-[~]
$ ping 192.168.189.10
PING 192.168.189.10 (192.168.189.10) 56(84) bytes of data.
64 bytes from 192.168.189.10: icmp_seq=1 ttl=64 time=2.01 ms
64 bytes from 192.168.189.10: icmp_seq=2 ttl=64 time=0.967 ms
64 bytes from 192.168.189.10: icmp_seq=3 ttl=64 time=0.823 ms
64 bytes from 192.168.189.10: icmp_seq=4 ttl=64 time=0.660 ms
^C
— 192.168.189.10 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 0.660/1.114/2.008/0.527 ms
```

Network Traffic Analysis Using Wireshark

Introduction:

Wireshark is a widely used network protocol analyzer that captures and inspects network traffic in real time. It enables network administrators and security analysts to examine data packets exchanged between devices, identify communication patterns, troubleshoot network issues, and detect suspicious activities. During this lab, Wireshark was used to monitor the communication between the virtual machines and observe different protocols operating at various layers of the TCP/IP model.

After installing Wireshark on the Windows host machine, the appropriate VMware network adapter was selected for packet capturing. Network traffic was generated by performing activities such as pinging different machines, accessing websites. The captured packets

were then filtered and analyzed to understand the functionality of various network protocols.

Protocol Analysis

During the packet capture session, the following protocols were successfully identified and analyzed.

Address Resolution Protocol (ARP):

The Address Resolution Protocol (ARP) operates at the Data Link Layer and is responsible for resolving an IPv4 address into its corresponding MAC address. Whenever a device wants to communicate with another device on the same local network, it first broadcasts an ARP request to determine the destination MAC address. The target device then responds with an ARP reply containing its hardware address.

Observation:

During the packet capture, ARP request and reply packets were observed whenever communication was initiated on browsers. These packets confirmed that MAC address resolution was taking place successfully before actual data transmission.

2	0.614244400	Intel_a2:42:75	HuaweiTechno_99:05:...	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1
3	0.617270500	HuaweiTechno_99:05:...	Intel_a2:42:75	ARP	42	192.168.1.1	61:75:99:05:b4
8911	62.615872700	Intel_a2:42:75	HuaweiTechno_99:05:...	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1
8912	62.618591500	HuaweiTechno_99:05:...	Intel_a2:42:75	ARP	42	192.168.1.1	61:75:99:05:b4
66255	184.109237200	HuaweiTechno_99:05:...	Broadcast	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1
66369	186.055359200	HuaweiTechno_99:05:...	Broadcast	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1
66656	187.298622800	HuaweiTechno_99:05:...	Broadcast	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1
66831	188.208288600	HuaweiTechno_99:05:...	Broadcast	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1
67004	190.358594500	HuaweiTechno_99:05:...	Broadcast	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1
67076	191.379535300	HuaweiTechno_99:05:...	Broadcast	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1
67298	192.403528500	HuaweiTechno_99:05:...	Broadcast	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1
68571	195.373647200	HuaweiTechno_99:05:...	Broadcast	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1
68925	196.406482900	HuaweiTechno_99:05:...	Broadcast	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1
68973	197.423356900	HuaweiTechno_99:05:...	Broadcast	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1
69037	198.449946000	HuaweiTechno_99:05:...	Broadcast	ARP	42	Who has 192.168.1.1	Tell 192.168.1.1

Transmission Control Protocol (TCP)

Transmission Control Protocol (TCP) is a reliable, connection-oriented transport layer protocol. It guarantees that data is delivered accurately and in the correct order through mechanisms such as acknowledgments, retransmissions, and flow control. TCP is commonly used by applications that require reliable communication, including web browsing, secure connections, and file transfers.

Observation:

The captured traffic showed the TCP three-way handshake consisting of SYN, SYN-ACK, and ACK packets before establishing a communication session. This confirmed that reliable connections were being established between communicating devices.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.1.10	192.168.1.1	TCP	66	55965 → 1515 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK PERM
6	1.299779700	192.168.1.10	64.233.170.100	TCP	55	51223 → 5228 [ACK] Seq=1 Ack=1 Win=253 Len=1
7	1.443746100	64.233.170.100	192.168.1.10	TCP	66	5228 → 51223 [ACK] Seq=1 Ack=2 Win=1046 Len=0 SLE=1 SRE=2
8	1.534453900	92.38.128.100	192.168.1.10	TCP	54	443 → 51731 [ACK] Seq=1 Ack=1 Win=501 Len=0
9	1.534540200	192.168.1.10	92.38.128.100	TCP	54	[TCP ACKed unseen segment] 51731 → 443 [ACK] Seq=1 Ack=2 Win=254 Len=0
10	1.557168800	192.168.1.10	92.38.128.100	TCP	55	[TCP Keep-Alive] 51731 → 443 [ACK] Seq=0 Ack=2 Win=254 Len=1
11	1.752214300	92.38.128.100	192.168.1.10	TCP	66	[TCP Previous segment not captured] 443 → 51731 [ACK] Seq=2 Ack=1 Win=5
12	4.041749000	192.168.1.10	110.93.100.100	TCP	54	62588 → 80 [FIN, ACK] Seq=1 Ack=1 Win=252 Len=0
13	4.188433000	110.93.100.100	192.168.1.10	TCP	54	80 → 62588 [FIN, ACK] Seq=1 Ack=2 Win=502 Len=0
14	4.188610500	192.168.1.10	110.93.100.100	TCP	54	62588 → 80 [ACK] Seq=2 Ack=2 Win=252 Len=0
29	5.192583000	172.64.1.100	192.168.1.10	TLSv1.2	78	Application Data
30	5.193591800	192.168.1.10	172.64.1.100	TLSv1.2	82	Application Data

User Datagram Protocol (UDP)

User Datagram Protocol (UDP) is a connectionless transport layer protocol that provides faster communication by transmitting packets without establishing a connection. Unlike TCP, UDP does not guarantee packet delivery or sequencing, making it suitable for applications where speed is more important than reliability.

Observation:

UDP packets were identified during DNS queries and other lightweight network services. The captured traffic demonstrated how UDP enables quick communication with minimal overhead.

No.	Time	Source	Destination	Protocol	Length	Info
4	0.630829600	192.168.1.10	142.251.142.100	UDP	71	61512 → 443 Len=29
5	0.692518800	142.251.142.100	192.168.1.10	UDP	66	443 → 61512 Len=24
15	4.764085600	192.168.1.10	172.64.1.100	UDP	1292	49281 → 443 Len=1250
16	4.764369300	192.168.1.10	172.64.1.100	UDP	1292	49281 → 443 Len=1250
17	4.764542000	192.168.1.10	172.64.1.100	UDP	1292	49281 → 443 Len=1250
18	4.764702200	192.168.1.10	172.64.1.100	UDP	1292	49281 → 443 Len=1250
19	4.765014900	192.168.1.10	172.64.1.100	UDP	1292	49281 → 443 Len=1250
20	4.765245800	192.168.1.10	172.64.1.100	UDP	1292	49281 → 443 Len=1250
21	4.765449300	192.168.1.10	172.64.1.100	UDP	802	49281 → 443 Len=760
22	4.765964600	192.168.1.10	172.64.1.100	UDP	1270	49281 → 443 Len=1228
23	4.809167400	172.64.1.100	192.168.1.10	UDP	66	443 → 49281 Len=24
24	4.809558200	172.64.1.100	192.168.1.10	UDP	66	443 → 49281 Len=24
25	4.814086800	172.64.1.100	192.168.1.10	UDP	66	443 → 49281 Len=24

Domain Name System (DNS)

The Domain Name System (DNS) is responsible for translating human-readable domain names into IP addresses. Before establishing communication with a remote server, a client first sends a DNS query to determine the corresponding IP address.

Observation:

DNS request and response packets were observed while accessing websites from the virtual environment. These packets demonstrated the process of hostname resolution before network communication could begin.

Time	Source	Destination	Protocol	Length	Info
40 7.469829300	192.168.	192.168.	DNS	80	Standard query 0x8cee HTTPS beacons.gcp.gvt2.com
41 7.471253800	192.168.	192.168.	DNS	80	Standard query 0x7dae A beacons.gcp.gvt2.com
42 7.478202900	192.168.	192.168.	DNS	176	Standard query response 0x7dae A beacons.gcp.gvt2.com CNAME beacons-handoff.g
43 7.481388100	192.168.	192.168.	DNS	193	Standard query response 0x8cee HTTPS beacons.gcp.gvt2.com CNAME beacons-handoff.g
77 8.477456800	192.168.	192.168.	DNS	78	Standard query 0xa1e9 HTTPS e2c10.gcp.gvt2.com
78 8.478301400	192.168.	192.168.	DNS	78	Standard query 0x5506 A e2c10.gcp.gvt2.com
81 8.487338700	192.168.	192.168.	DNS	135	Standard query response 0xa1e9 HTTPS e2c10.gcp.gvt2.com SOA ns1.google.com
82 8.499550000	192.168.	192.168.	DNS	94	Standard query response 0x5506 A e2c10.gcp.gvt2.com A 34.116.74.210
203 12.786345700	192.168.	192.168.	DNS	74	Standard query 0x530e HTTPS lh3.google.com
204 12.786768100	192.168.	192.168.	DNS	74	Standard query 0x4b2a A lh3.google.com

Transport Layer Security (TLS)

Transport Layer Security (TLS) is a cryptographic protocol that provides confidentiality, integrity, and authentication for data transmitted across a network. It encrypts communication between clients and servers to protect sensitive information from unauthorized access.

Observation:

TLS packets were observed during secure web browsing activities. The packet capture showed the TLS handshake process, indicating that encrypted communication had been successfully established between the client and the remote server.

Time	Source	Destination	Protocol	Length	Info
57 7.961655100	192.168.	35.209.	TLSv1.3	696	Client Hello (SNI=beacons.gcp.gvt2.com)
61 8.226143400	35.209.	192.168.	TLSv1.3	1466	Server Hello, Change Cipher Spec
64 8.226157500	35.209.	192.168.	TLSv1.3	332	Application Data
66 8.231262000	192.168.	35.209.	TLSv1.3	118	Change Cipher Spec, Application Data
67 8.232449600	192.168.	35.209.	TLSv1.3	146	Application Data
68 8.233166100	192.168.	35.209.	TLSv1.3	301	Application Data
69 8.233452700	192.168.	35.209.	TLSv1.3	354	Application Data
70 8.461438900	35.209.	192.168.	TLSv1.3	672	Application Data, Application Data
72 8.461443800	35.209.	192.168.	TLSv1.3	85	Application Data
74 8.463427300	192.168.	35.209.	TLSv1.3	85	Application Data
75 8.474149900	35.209.	192.168.	TLSv1.3	895	Application Data
79 8.479809400	192.168.	35.209.	TLSv1.3	93	Application Data
80 8.480063400	192.168.	35.209.	TLSv1.3	89	Application Data
89 8.725703700	192.168.	34.116.	TLSv1.3	370	Client Hello (SNI=e2c10.gcp.gvt2.com)
91 11.440411000	192.168.	4.213.	TLSv1.2	157	Application Data

Wazuh Agent Deployment

After successfully creating the Wazuh VM in Virtual Box, the next step was to deploy Wazuh agents on the monitored endpoints. Wazuh agents act as lightweight monitoring components that collect system logs, security events, and endpoint information before securely forwarding them to the Wazuh Manager.

In this laboratory environment, the Windows 11 host machine and the Kali Linux virtual machine were configured as monitored endpoints. Both systems were enrolled with the Wazuh server to enable centralized log collection and continuous security monitoring.

The deployment process involved installing the Wazuh agent, specifying the Wazuh Manager's IP address, registering the agent, and verifying successful communication with

the server. Once connected, the agents automatically started sending endpoint information and security events to the Wazuh Dashboard.

The successful deployment of these agents transformed the laboratory into a centralized Security Information and Event Management (SIEM) environment, where multiple endpoints could be monitored from a single dashboard.

WindowsAgentDeployment

The Wazuh agent was first deployed on the Windows 11 system. During the installation process, the IP address of the Wazuh Manager was specified so that the agent could establish communication with the server. The agent was also assigned a unique hostname to simplify endpoint identification within the dashboard.

Once the installation was completed, the Wazuh service was started, allowing the Windows endpoint to register with the Wazuh Manager and begin transmitting system logs and security events.

```
Windows PowerShell 5.1
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\chfar> wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.13.1-1_amd64.deb && sud
o WAZUH_MANAGER='192.168.189.131' dpkg -i ./wazuh-agent_4.13.1-1_amd64.deb
```

KaliLinuxAgentDeployment

Following the Windows deployment, the Wazuh agent was installed on the Kali Linux virtual machine. The manager's IP address was configured within the agent settings, allowing the endpoint to establish secure communication with the Wazuh server.

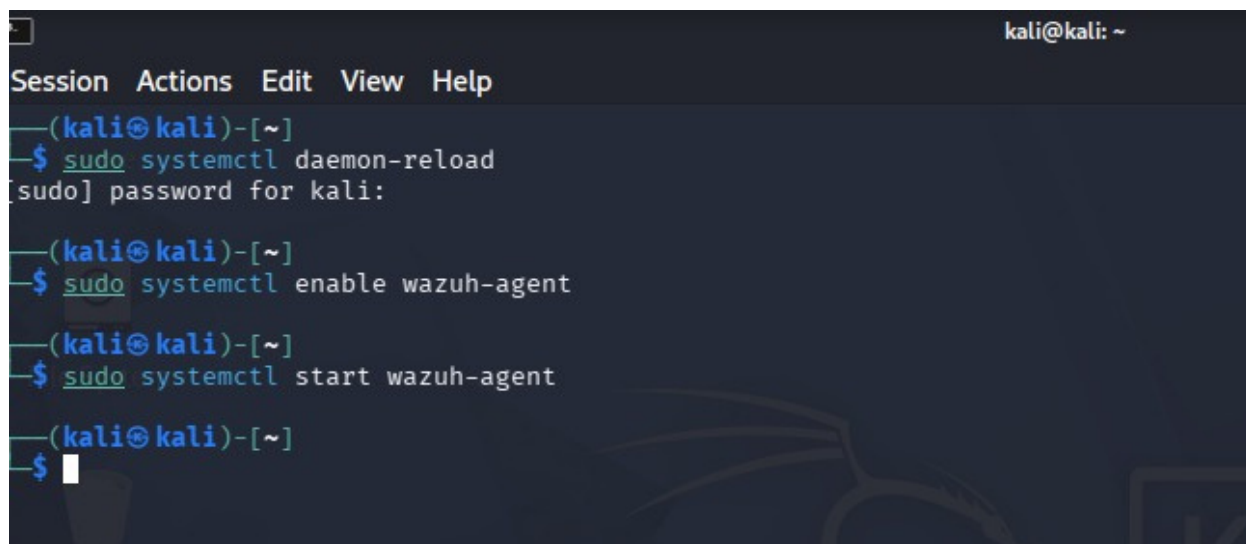
After completing the installation, the Wazuh agent service was enabled and started. This ensured that the service would automatically start whenever the operating system boots and continuously forward endpoint events to the Wazuh Manager.

```
(kali@kali)~$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.13.1-1_amd64.deb && sudo WAZUH_MANAGER='192.168.189.131' dpkg
-i ./wazuh-agent4.13.1-1amd64.deb
```

Startingthe WazuhAgentonKaliLinux

After installing the Wazuh agent on the Kali Linux virtual machine, the service was enabled and started using the systemctl utility. Enabling the service ensured that the agent would automatically start whenever the operating system boots, while starting the service immediately established communication with the Wazuh Manager.

This step successfully activated the Kali Linux agent, allowing it to begin collecting endpoint information and forwarding security events to the Wazuh server.

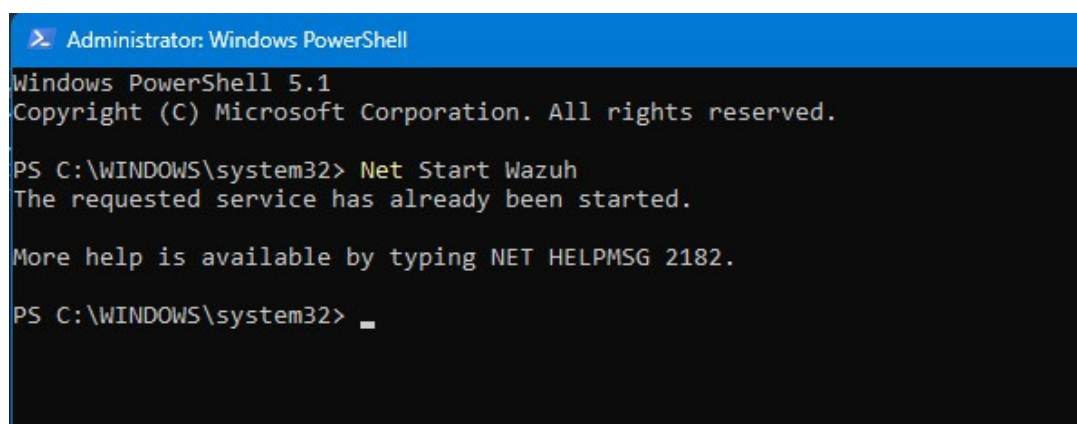
A terminal window on a Kali Linux system. The window title is 'kali@kali: ~'. The menu bar shows 'Session Actions Edit View Help'. The terminal shows the following commands and output:

```
(kali@kali)-[~]  
$ sudo systemctl daemon-reload  
[sudo] password for kali:  
  
(kali@kali)-[~]  
$ sudo systemctl enable wazuh-agent  
  
(kali@kali)-[~]  
$ sudo systemctl start wazuh-agent  
  
(kali@kali)-[~]  
$
```

Starting the Wazuh Agent on Windows

Following the installation of the Wazuh agent on the Windows system, the Windows Service Manager was used to verify that the Wazuh service was running correctly. The command output confirmed that the service had already been started, indicating that the agent was active and functioning properly.

With the service running successfully, the Windows endpoint was able to send logs and security events to the Wazuh server for centralized monitoring.

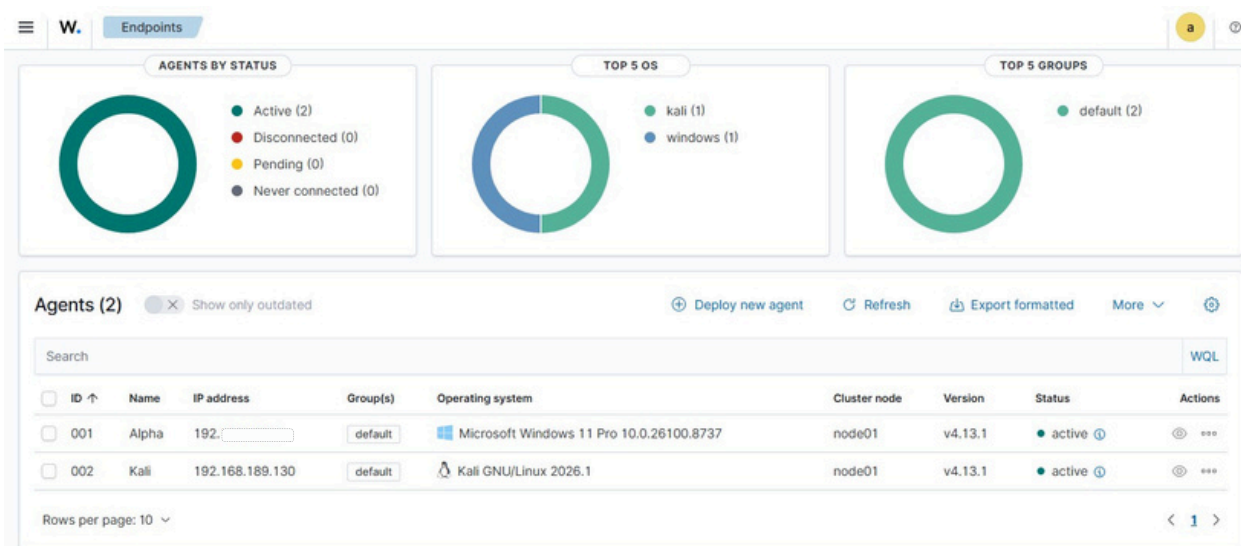
A screenshot of a Windows PowerShell window titled 'Administrator: Windows PowerShell'. The window shows the following text:

```
Windows PowerShell 5.1  
Copyright (C) Microsoft Corporation. All rights reserved.  
  
PS C:\WINDOWS\system32> Net Start Wazuh  
The requested service has already been started.  
  
More help is available by typing NET HELPMSG 2182.  
  
PS C:\WINDOWS\system32> _
```

Verification of Connected Agents

After configuring both endpoints, the Wazuh Dashboard was used to verify their connection status. The dashboard displayed both the Windows and Kali Linux agents as **Active**, confirming successful enrollment and continuous communication with the Wazuh Manager.

The dashboard also provided additional information, including the operating system, IP address, agent version, and current health status of each endpoint. This verification confirmed that the SIEM infrastructure was functioning correctly and was capable of monitoring multiple endpoints simultaneously.



Security Monitoring and Threat Detection

After successfully deploying the Wazuh SIEM server and enrolling both Windows and Kali Linux agents, the final phase of the lab focused on validating the monitoring capabilities of the environment. To generate security events, an Nmap scan was performed from the Kali Linux machine against the Windows endpoint. The objective was to observe how network reconnaissance activities are detected and recorded by the Wazuh platform.

This exercise demonstrated how a SIEM solution collects endpoint logs, correlates security events, and presents them through a centralized dashboard, allowing security analysts to identify potentially suspicious activities in real time.

NmapPortScan

An Nmap SYN scan was initiated from the Kali Linux machine to identify open ports on the Windows endpoint. The scan confirmed that the target host was reachable and successfully identified one open TCP port while filtering the remaining ports.

This initial scan verified network connectivity and demonstrated how reconnaissance activities can reveal exposed network services before any further security assessment.

```
[sudo] password for kali:
(root@kali)-[/home/kali]
# nmap -sS 192.168.189.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-26 09:22 -0400
Stats: 0:00:03 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 0.50% done
Stats: 0:00:04 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 0.50% done
Stats: 0:00:17 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 63.55% done; ETC: 09:22 (0:00:08 remaining)
Nmap scan report for 192.168.189.1
Host is up (0.00095s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
7070/tcp  open  realserver
MAC Address: 00:50:56:00:00:00 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 18.86 seconds
```

Figure shows the results of the Nmap SYN scan performed from Kali Linux against the Windows endpoint. The scan confirmed that the target host was online and identified an open TCP service while filtering the remaining ports.

Service and Operating System Detection

To gather additional information about the target system, an advanced Nmap scan was performed using service and operating system detection. The scan identified the running service on the open port and provided an operating system fingerprint, indicating that the target machine was running Microsoft Windows.

This type of reconnaissance is commonly used during the information-gathering phase of penetration testing to understand the services exposed by a target system before conducting further security assessments.

```

(root@kali)-[/home/kali]
└─$ nmap -A 192.168.1.100
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-26 09:23 -0400
Stats: 0:00:38 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 0.00% done
Nmap scan report for 192.168.1.100
Host is up (0.00055s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
7070/tcp  open  ssl/realserver?
| ssl-cert: Subject: commonName=AnyDesk Client
| Not valid before: 2025-12-07T11:42:33
|_ Not valid after: 2075-11-25T11:42:33
|_ ssl-date: TLS randomness does not represent time
MAC Address: 00:50:56:C0:00:01 (VMware)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose|phone
Running (JUST GUESSING): Microsoft Windows 11|10|2022|2008|Phone (96%)
OS CPE: cpe:/o:microsoft:windows_11 cpe:/o:microsoft:windows_10 cpe:/o:microsoft:windows_server_2022 cpe:/o:microsoft:windows_server_2008::sp
1 cpe:/o:microsoft:windows
Aggressive OS guesses: Microsoft Windows 11 21H2 (96%), Microsoft Windows 10 (91%), Microsoft Windows 10 1607 (91%), Microsoft Windows Server
2022 (90%), Microsoft Windows Server 2008 SP1 (88%), Microsoft Windows Phone 7.5 or 8.0 (88%), Microsoft Windows 10 1511 - 1607 (86%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

TRACEROUTE
HOP RTT      ADDRESS
1   0.55 ms  192.168.1.100

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 50.07 seconds

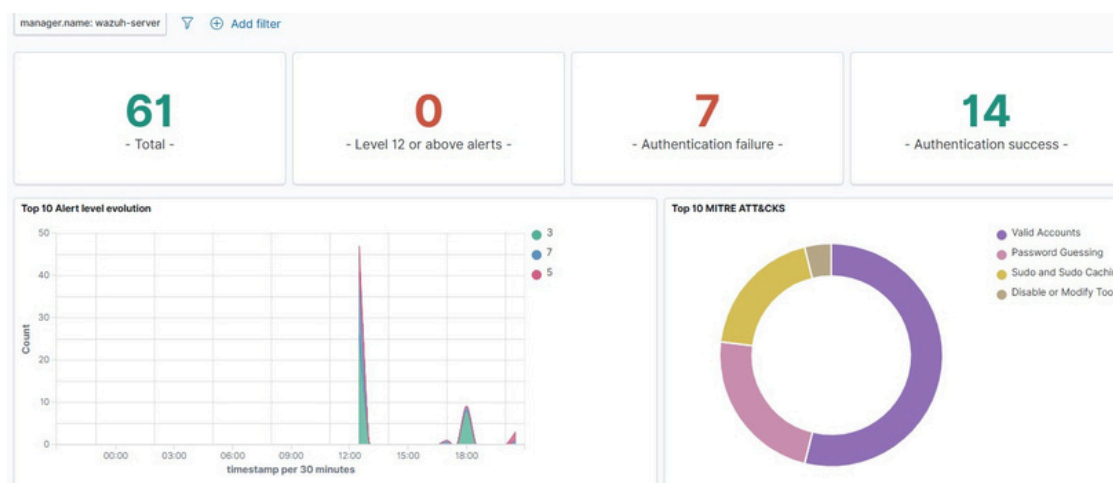
```

Figure presents the results of the advanced Nmap scan, including service identification and operating system detection. The scan successfully detected the active service and generated an operating system fingerprint based on the observed network responses.

Monitoring Security Events in Wazuh

After the reconnaissance activity was completed, the Wazuh Dashboard was used to monitor the generated security events. The dashboard collected logs from the connected endpoints and displayed authentication events, alert statistics, and MITRE ATT&CK mappings.

The monitoring interface confirmed that the Windows and Kali Linux agents were actively communicating with the Wazuh Manager. The dashboard also provided a graphical overview of authentication events and security alerts, demonstrating the platform's capability to centralize endpoint monitoring and support security analysis.



MITRE ATTACK Mapping and Threat Hunting

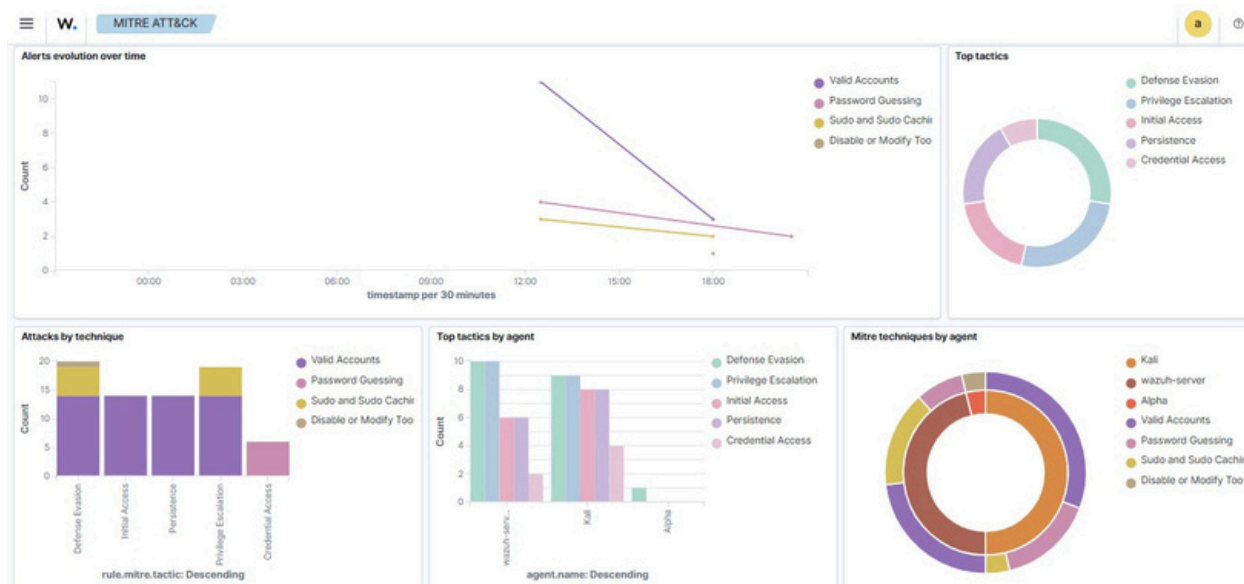
After collecting security events from the connected endpoints, Wazuh automatically correlated the detected activities with the MITRE ATT&CK framework. This feature enables security analysts to understand attacker behavior by mapping alerts to standardized adversarial tactics and techniques. Instead of viewing raw log data, analysts can identify which attack techniques were used and determine the stage of the cyber kill chain.

In this laboratory, the alerts generated from both the Windows and Kali Linux endpoints were successfully classified according to MITRE ATT&CK, providing a structured and industry-recognized approach to threat analysis.

MITRE ATT&CK Dashboard

The MITRE ATT&CK dashboard presented a visual overview of the security events detected throughout the laboratory. The dashboard categorized alerts into different attack tactics such as Credential Access, Privilege Escalation, Defense Evasion, Persistence, and Initial Access.

The graphical representation also showed which endpoint generated the highest number of alerts, allowing analysts to quickly identify the most active systems and the techniques associated with them.



MITRE ATT&CK Event Correlation

Each detected event was automatically mapped to its corresponding MITRE ATT&CK technique. For example, failed login attempts were associated with **Credential Access (T1110.001)**, while successful privilege escalation through **sudo** was mapped to **Privilege Escalation (T1548.003)**. Login session activities were also linked to techniques related to **Defense Evasion, Persistence, and Privilege Escalation**.

This automatic correlation helps SOC analysts quickly understand the intent behind suspicious activities without manually analyzing individual log entries.

MITRE ATT&CK							
Jun 25, 2026 @ 21:01:40.055 - Jun 26, 2026 @ 21:01:40.055							
Export Formatted Reset view 605 available fields Columns Density 1 fields sorted Full screen							
timestamp	agent.name	rule.mitre.id	rule.mitre.tactic	rule.description	rule.level	rule.id	
Jun 26, 2026 @ 20:45:30.2...	Kali	T1110.001	Credential Access	PAM: User login failed.	5	5503	
Jun 26, 2026 @ 20:45:30.2...	Kali	T1110.001	Credential Access	unix_chkpwd: Password check failed.	5	5557	
Jun 26, 2026 @ 18:20:45.7...	Kali	T1078	Defense Evasion, Persistence, Privileg...	PAM: Login session opened.	3	5501	
Jun 26, 2026 @ 18:20:45.7...	Kali	T1078	Defense Evasion, Persistence, Privileg...	PAM: Login session opened.	3	5501	
Jun 26, 2026 @ 18:20:45.7...	Kali	T1548.003	Privilege Escalation, Defense Evasion	Successful sudo to ROOT executed.	3	5402	
Jun 26, 2026 @ 18:14:59.5...	Alpha	T1562.001	Defense Evasion	Wazuh agent stopped.	3	506	
Jun 26, 2026 @ 18:12:56.7...	wazuh-server	T1078	Defense Evasion, Persistence, Privileg...	PAM: Login session opened.	3	5501	
Jun 26, 2026 @ 18:12:56.7...	wazuh-server	T1548.003	Privilege Escalation, Defense Evasion	Successful sudo to ROOT executed.	3	5402	
Jun 26, 2026 @ 12:56:55.7...	Kali	T1078	Defense Evasion, Persistence, Privileg...	PAM: Login session opened.	3	5501	
Jun 26, 2026 @ 12:56:53.6...	Kali	T1078	Defense Evasion, Persistence, Privileg...	PAM: Login session opened.	3	5501	
Jun 26, 2026 @ 12:56:53.6...	Kali	T1078	Defense Evasion, Persistence, Privileg...	PAM: Login session opened.	3	5501	
Jun 26, 2026 @ 12:49:33.2...	wazuh-server	T1078	Defense Evasion, Persistence, Privileg...	PAM: Login session opened.	3	5501	
Jun 26, 2026 @ 12:49:33.2...	wazuh-server	T1548.003	Privilege Escalation, Defense Evasion	Successful sudo to ROOT executed.	3	5402	
Jun 26, 2026 @ 12:49:27.2...	wazuh-server	T1078	Defense Evasion, Persistence, Privileg...	PAM: Login session opened.	3	5501	
Jun 26, 2026 @ 12:49:27.2...	wazuh-server	T1548.003	Privilege Escalation, Defense Evasion	Successful sudo to ROOT executed.	3	5402	

MITRE ATT&CK Correlation Analysis

MITRE Technique ID	Technique	Detected Activity
T1078	Valid Accounts	Successful user login sessions
T1110.001	Password Guessing	Failed login attempts
T1548.003	Abuse Elevation Control Mechanism	Successful execution of sudo commands
T1562.001	Disable or Modify Tools	Wazuh agent service stopped
T1046 (if Nmap detection is enabled)	Network Service Discovery	Network reconnaissance using Nmap

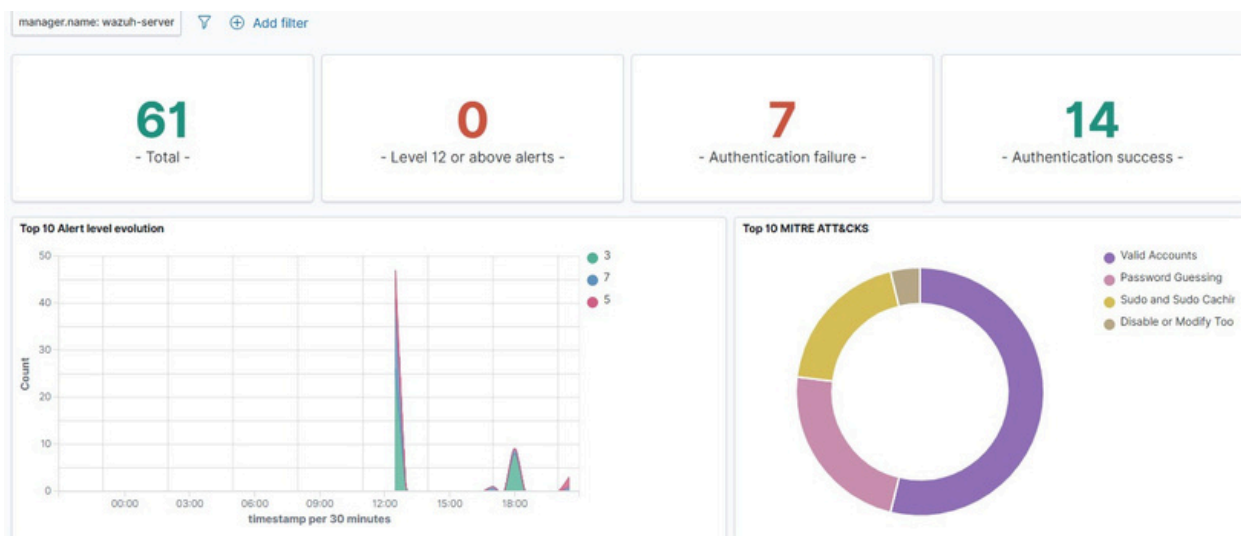
Threat Hunting

Wazuh's ThreatHuntingmodule was used to examine the collected security events in greater detail. Each event included information such as the timestamp, monitored endpoint, rule description, severity level, and unique rule identifier.

During the analysis, several security-related activities were observed, including:

- User login failures
- Successful user login sessions
- Privilege escalation using **sudo**
 - Wazuh agent start and stop events
 - Network port status changes detected through Netstat

These events demonstrate how Wazuh continuously monitors endpoint activity and records security-relevant behavior for further investigation.



ThreatHuntingEventCorrelation

The Threat Hunting module was used to analyze security events collected from the monitored endpoints. Wazuh continuously gathered logs from the Windows host, Kali Linux machine, and the Wazuh server, allowing all security-related activities to be viewed from a centralized interface.

During the analysis, each alert was correlated with its originating endpoint, associated security rule, and severity level. This correlation enabled rapid identification of suspicious activities and provided analysts with sufficient context to understand the sequence of events.

The Threat Hunting interface displayed multiple security events, including authentication failures, successful login sessions, privilege escalation using **sudo**, Wazuh agent status changes, and network port modifications. These events were automatically organized based on their timestamps, monitored hosts, and predefined Wazuh detection rules.

W. Threat Hunting					
Export Formatted Reset View 605 available fields Columns Density 1 fields sorted Full screen					
timestamp	agent.name	rule.description	rule.level	rule.id	
Jun 26, 2026 @ 20:45:30.2...	Kali	PAM: User login failed.	5	5503	
Jun 26, 2026 @ 20:45:30.2...	Kali	unix_chkpwd: Password check failed.	5	5557	
Jun 26, 2026 @ 20:40:12.6...	Kali	Listened ports status (netstat) changed (new port opened or closed).	7	533	
Jun 26, 2026 @ 18:20:45.7...	Kali	PAM: Login session opened.	3	5501	
Jun 26, 2026 @ 18:20:45.7...	Kali	PAM: Login session opened.	3	5501	
Jun 26, 2026 @ 18:20:45.7...	Kali	Successful sudo to ROOT executed.	3	5402	
Jun 26, 2026 @ 18:15:27.1...	Alpha	Wazuh agent started.	3	503	
Jun 26, 2026 @ 18:14:59.5...	Alpha	Wazuh agent stopped.	3	506	
Jun 26, 2026 @ 18:13:02.8...	wazuh-server	PAM: Login session closed.	3	5502	
Jun 26, 2026 @ 18:12:56.7...	wazuh-server	PAM: Login session opened.	3	5501	
Jun 26, 2026 @ 18:12:56.7...	wazuh-server	Successful sudo to ROOT executed.	3	5402	
Jun 26, 2026 @ 18:09:41.2...	Kali	Listened ports status (netstat) changed (new port opened or closed).	7	533	
Jun 26, 2026 @ 17:18:29.5...	Kali	Listened ports status (netstat) changed (new port opened or closed).	7	533	
Jun 26, 2026 @ 13:08:33.6...	Kali	PAM: Login session closed.	3	5502	
Jun 26, 2026 @ 12:58:16.3...	wazuh-server	CIS Benchmark for Amazon Linux 2023 Benchmark v1.0.0.: Ensure sticky bit is set on a...	3	19009	

Threat Hunting Correlation Analysis

Event	Endpoint	Rule Level	Description
Failed User Login	Kali Linux	5	Multiple failed authentication attempts were detected through the PAM authentication module.
Password Verification Failure	Kali Linux	5	Wazuh recorded unsuccessful password validation attempts generated by the Unix authentication service.
Network Port Status Change	Kali Linux	7	Netstat detected changes in listening network ports, indicating that services were started or stopped.
Successful Login Session	Kali Linux / Wazuh Server	3	Successful user authentication and session establishment were recorded.
Privilege Escalation	Kali Linux / Wazuh Server	3	Execution of sudo commands was detected, indicating privileged administrative activity.
Wazuh Agent Started	Windows Endpoints	3	The Windows endpoint successfully initialized and connected its Wazuh agent

Analysis

TheThreat Hunting results confirmed that Wazuh successfully correlated security events generated across all monitored endpoints. Authentication activities, privilege escalation events, service status changes, and network-related modifications were collected and organized into a centralized timeline. This correlation enabled efficient investigation of endpoint activities while providing valuable context for incident analysis and threat detection.

The combination of event timestamps, rule descriptions, severity levels, and endpoint information demonstrates how Wazuh supports Security Operations Center (SOC) analysts in identifying suspicious behavior and conducting forensic investigations through centralized log analysis.

LearningOutcomes

Upon completion of this laboratory, the following learning outcomes were achieved:

- Successfully designed and deployed a virtual cybersecurity laboratory using VMware Workstation.
- Configured NAT and Host-Only virtual networks for secure communication.
- Assigned and verified static IP addressing for the Wazuh server.
- Analyzed network traffic using Wireshark and identified common network protocols.
- Installed and configured the Wazuh SIEM platform on Ubuntu Server.
- Successfully deployed Wazuh agents on Windows and Kali Linux endpoints.
- Verified communication between the Wazuh Manager and connected agents.
- Performed network reconnaissance using Nmap.
- Monitored endpoint activities through the Wazuh Dashboard.
- Investigated security events using the Threat Hunting module.
- Correlated security alerts with the MITRE ATT&CK framework.
- Gained practical experience in centralized log management, security monitoring, and incident analysis.